CinemaScope

Interview Defence Guide

PySpark • Databricks • Delta Lake • Bronze / Silver / Gold • SQL • Data Quality

Justify every resume claim. Explain every term. Answer follow-ups with evidence.

What this guide is

A rebuilt edition of your CinemaScope question bank. It is organised around what an interviewer will actually do: point at a line on your resume, ask you to prove it, then ask what a word in it means. Every number below is copied from your notebook's saved outputs, and every claim is marked **supported**, **partly supported** or **needs a fix**.

What changed from the earlier version

- **The 350,000 claim is now verified.** Your final SQL cell did save its output: **349,988** movie records. The old guide said it was missing.
- **11 diagrams** replace walls of text: pipeline, layers, join key, formula, quality checks, SQL order and more.
- **Duplicate questions removed** (Q13 to Q18 appeared twice) and the Q&A regrouped by theme.
- **New honest weak spots** found by reading your outputs (for example how the vote threshold was chosen), each with a ready answer.
- **A ready-to-paste schema check** so the "schema consistency" claim becomes true.

Contents

1. Your resume claims, one by one . . . 3
2. Your project in 30 seconds, 90 seconds, and "why is this data engineering?" . . . 4
3. Architecture in pictures . . . 6
4. The code, explained so you can talk through it . . . 8
5. The weighted rating: your most distinctive idea . . . 11
6. Results you can quote in the interview . . . 13
7. The SQL layer . . . 16
8. Data quality: exactly what you check, and what you do not . . . 19
9. Terminology cards . . . 21
10. Question bank by theme . . . 24
11. Trap questions and cross-questions . . . 29
12. Revision card and pre-interview checklist . . . 30

READING ORDER

Short on time? Read in this order: Section 1 (claims) → Section 2 (your pitch) → Section 5 (weighted rating) → Section 8 (quality checks) → Section 11 (traps) → Section 12 (revision card). That is about 30 minutes and covers 90% of what an interviewer is likely to probe.

ABOUT THE RESUME

One note on sources. I worked from your notebook and the earlier question bank, which quotes your resume wording. I did not have the resume file itself. If a bullet on your resume is worded differently from the claims quoted in Section 1, map it by meaning and use the same evidence.

1. Your resume claims, one by one

For each claim: how strong it is, where the proof sits in the notebook, and the sentence to say. If an interviewer points at a phrase, answer with the exact implementation, not the broader idea you intended.

Resume claim	Verdict	Proof in the notebook	Say this
End-to-end PySpark pipeline in Databricks	**Supported**	Sections 1 to 9: reads TSV.GZ, writes Delta layers, registers tables, runs SQL, saves quality results.	"A batch pipeline in a Databricks notebook, from raw files to validated, queryable tables." Do not imply a scheduler.
Python, SQL, PySpark, Databricks	**Supported**	PySpark cells for ingestion and transforms; `%sql` cells for analytics; Unity Catalog volume and tables.	"PySpark for transformations, SQL for analytics, Databricks as the managed Spark environment."
Bronze, Silver and Gold layers	**Supported**	Ratings go Bronze → Silver → Gold. Gold also holds movie rankings and director stats. Basics and crew are read straight into Gold.	"Ratings pass through all three layers. Metadata and crew are joined in at Gold." Know that nuance before they ask.
Processed 350,000 movie records	**Supported**	Final summary SQL: **movie_records = 349,988**, equal to unique_titles. This is rated movies after joining ratings to movie metadata.	"About 350 thousand rated movie records, 349,988 exactly." Do not confuse it with 1,719,798 (all rated title types).
Five data-quality checks	**Supported**	Cell "Calculate Data Quality Metrics": null IDs, duplicate IDs, rating range, vote counts, null weighted ratings. All PASS, saved to a Delta table.	Name all five, in that order. Say they are rule checks on the Gold movie table.
Missing values	**Supported**	Null checks on tconst, rating, votes, weightedRating; `nullif` and `try_cast` convert the IMDb \N marker to NULL.	"I check for nulls in the key and score columns, and I convert IMDb's \N marker to real NULLs."
Duplicate records	**Supported**	`dropDuplicates(['tconst'])` in Silver; duplicate-ID check on the Gold table (0 failures).	"I deduplicate on the title ID in Silver and verify uniqueness again at Gold."
Schema consistency	**Needs a fix**	Explicit schemas are declared at read, and types are cast in Silver. But **no check compares actual schema to expected**.	Add check 6 (code in Section 8), or reword the bullet to "enforced explicit schemas at ingestion". Until then, say exactly that.
SQL to derive insights	**Supported**	Four queries: top movies, genre comparison, decade comparison, directors by movie count.	Pick one (genre query) and explain WHERE, GROUP BY, HAVING, ORDER BY.
Curated datasets for analytics	**Supported**	Gold Delta datasets saved, then registered as `workspace.default.cinemascope_*` tables.	"Gold tables are reusable: any SQL user can query them by name."

VERIFIED

The 350,000 figure is real. The final summary query returned 349,988 rows and 349,988 distinct titles, so no duplicates. Say "about 350,000" only if you also know the exact number. Be ready for "which dataset?": it is movie-type titles that have an IMDb rating, not every movie IMDb has ever listed.

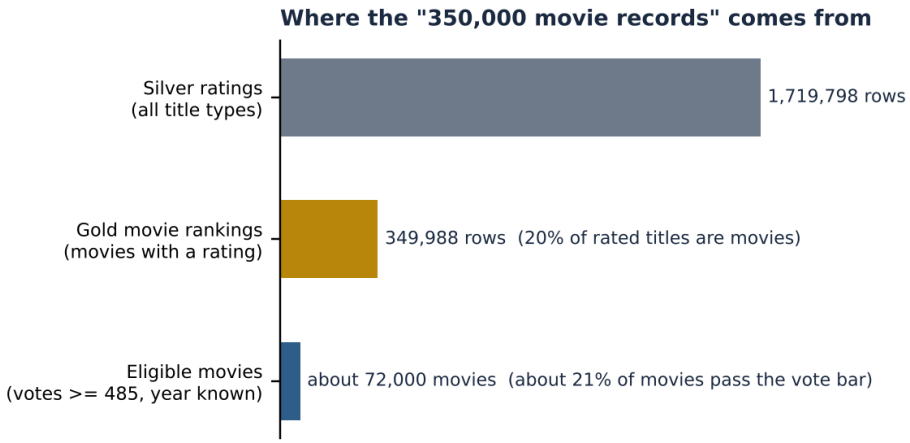


Figure 1. *Row counts at each stage. The 350K figure is the middle bar. The jump from 1.7M to 350K happens because ratings cover every title type (episodes, shorts and so on), while the pipeline keeps only movie.*

OPTIONAL REWORD

Airtight bullet, if you want to reword. "Built an end-to-end PySpark and Delta Lake pipeline on Databricks (Bronze, Silver, Gold) over IMDb bulk data, producing ~350K rated-movie records; applied five data-quality checks (null IDs, duplicate IDs, rating range, vote counts, null weighted scores) with explicit ingestion schemas; used SQL on curated Gold tables for genre, decade and director analytics." Every phrase in it is backed by a cell.

2. Your project in 30 seconds, 90 seconds, and "why is this data engineering?"

Practise these out loud until they sound like you. Short first, longer only if they lean in.

The 30-second version

SAY THIS

"CinemaScope is a batch pipeline I built on Databricks with PySpark, Delta Lake and SQL. It takes IMDb's raw ratings, title and crew files, cleans and layers them Bronze, Silver, Gold, scores every movie with a vote-adjusted rating, and serves the results as SQL tables. I also run five data-quality checks and save their results. The end product is about 350 thousand rated movies ready for analysis."

The 90-second version

SAY THIS

"The problem: raw IMDb data is compressed, tab-separated, uses a backslash-N marker for missing values, and ranking by raw average rating is misleading because a film with ten perfect votes beats a classic with millions.

So I ingest the three files with explicit schemas and save the ratings as a Bronze Delta layer. In Silver, I drop duplicate title IDs and filter invalid IDs, ratings outside 0 to 10 and negative votes. In Gold, I compute a Bayesian-style weighted rating that blends each title's rating with the global average of 6.97, using the 90th-percentile vote count of 485 as the weight. I join that to movie metadata, which gives 349,988 movie records, then explode genres and directors to produce genre, decade and director statistics. Everything is persisted as Delta, registered as tables, and queried with SQL. Five quality checks run on the final table and are saved for review."

"Why is this data engineering and not just analysis?"

> **SAY THIS**
> "The analysis is the consumer. The engineering is the ingestion with explicit schemas, the layered storage, the cleaning rules, the joins on a shared key, the persisted reusable tables, and the validation that runs before anyone trusts the numbers."

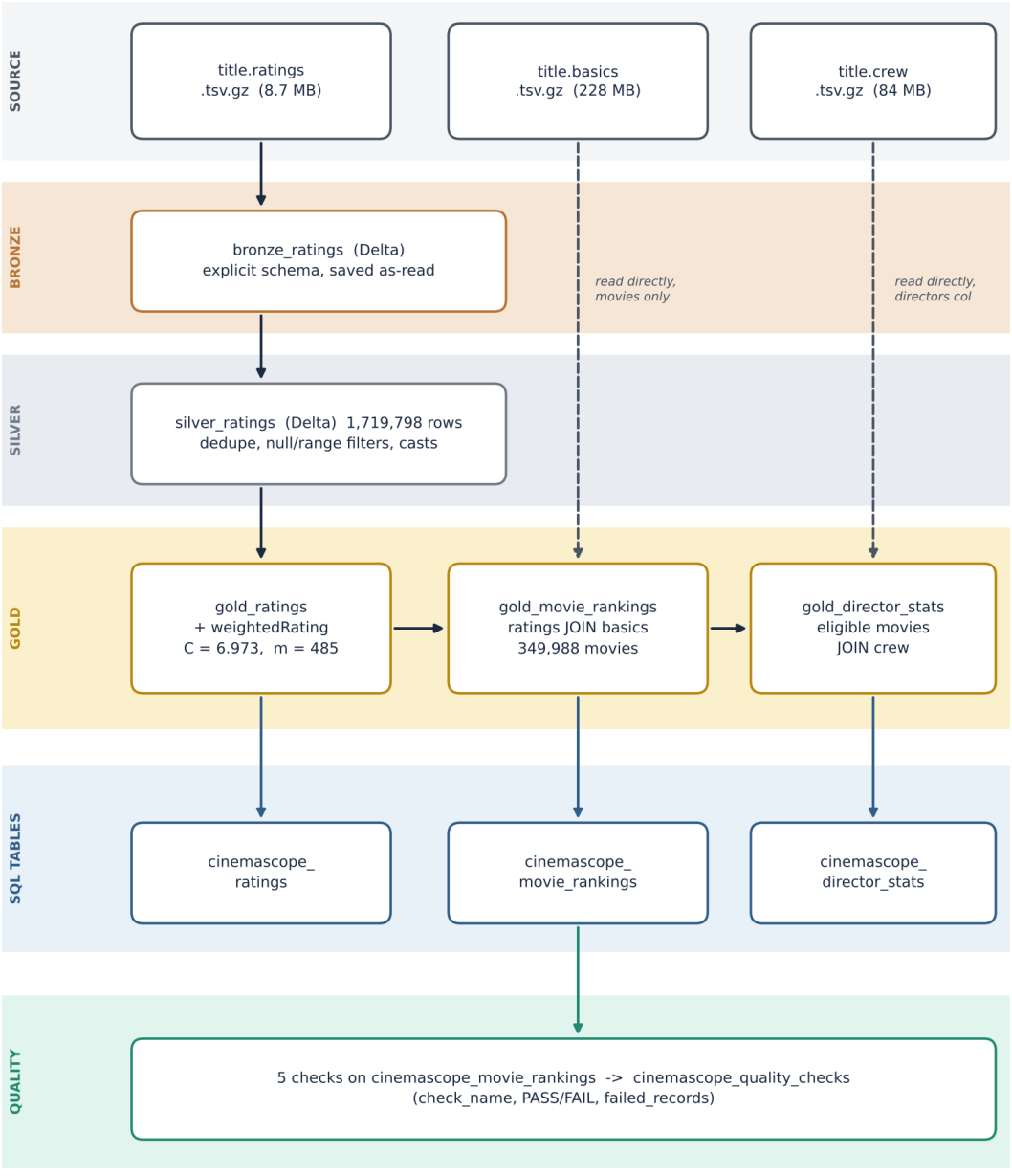
Say plainly what the notebook does not show

Do not claim	Because the notebook has no
Airflow, Workflows or any scheduler	scheduling or orchestration code
Streaming or real-time ingestion	readStream, checkpointing or triggers; everything is batch
Alerting, retries, CI/CD, monitoring	failure handling or deployment code
Schema-drift detection	check comparing actual schema with expected (see Section 8 to add one)
Delta time travel, MERGE, OPTIMIZE	any use of those features (it only writes and reads Delta)
Spark performance tuning or benchmarks	partitioning, caching, broadcast hints or timing comparisons

Being exact here earns trust. "I did not build that, here is how I would" beats a claim that collapses on the next question.

3. Architecture in pictures

Learn this flow so you can draw it on a whiteboard in under a minute: source files, three layers, SQL tables, quality checks.



Dashed lines: basics and crew have no Bronze/Silver copy in this notebook (be ready to say so).

Figure 2. *End-to-end flow. Solid arrows are the main path. Dashed arrows show basics and crew skipping Bronze and Silver and joining at Gold.*

What each layer means, in one picture

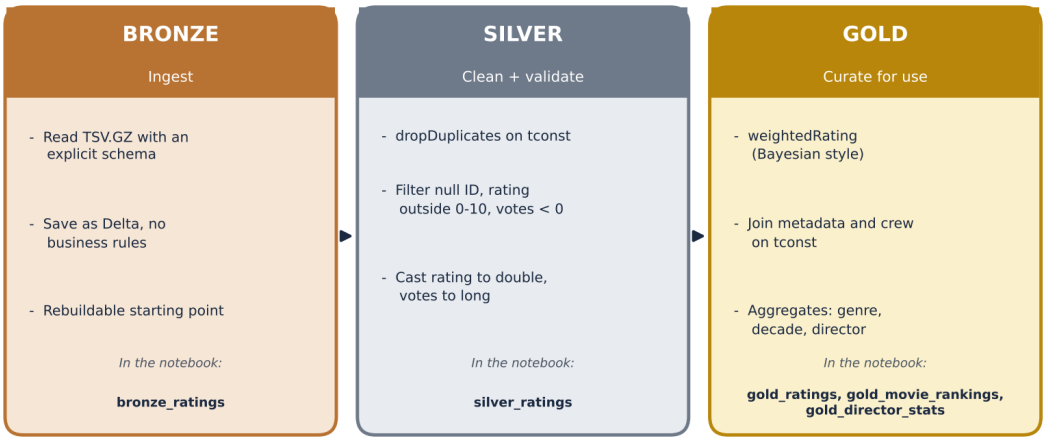


Figure 3. *Bronze keeps what was ingested, Silver makes it trustworthy, Gold makes it useful. These are patterns, not rigid rules.*

The key that ties everything together

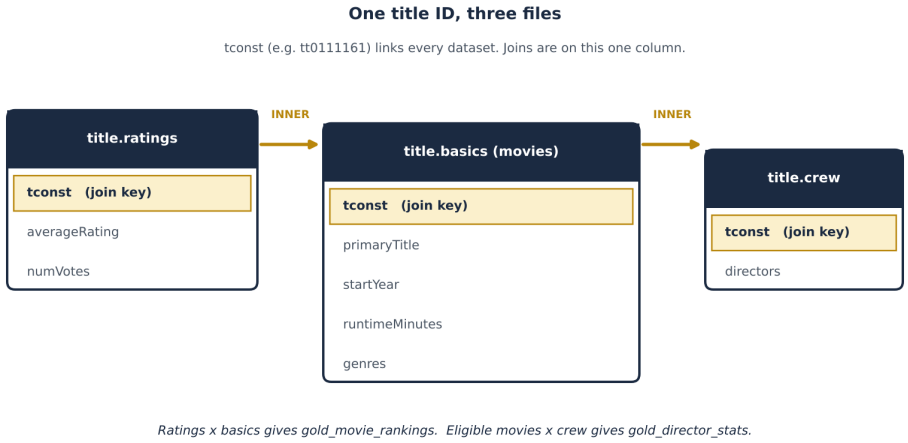


Figure 4. *Every IMDb file is keyed by* `tconst` *(the title ID). All joins in the project use that one column.*

Stage-by-stage map

Stage	Notebook section	What happens	Output
Source	1. Setup	Create Unity Catalog volume, list files	3 TSV.GZ files in `cinemascope_raw`
Bronze	2. Ratings pipeline	Read ratings with explicit schema, write Delta	`bronze_ratings`
Silver	2. Ratings pipeline	Dedupe, filter, cast, profile (1,719,798 rows)	`silver_ratings`
Gold: ratings	2 and 3	C = 6.973, m = 485, weightedRating	`gold_ratings`
Gold: movies	4. Movie metadata	Clean \N markers, inner join on tconst	`gold_movie_rankings` (349,988)
Gold: genre, decade	4 and 5	Explode genres, floor years to decades, aggregate	Result tables (shown, not saved)
Gold: directors	6. Director analytics	Join crew, explode directors, keep ≥ 3 movies	`gold_director_stats`
SQL tables	7 and 8	saveAsTable, then four SQL analyses	`workspace.default.cinemascope_*`
Quality	9	Five checks, save results	`cinemascope_quality_checks`

4. The code, explained so you can talk through it

Each block shows the real notebook code (trimmed), what it does line by line, and what to say. If an interviewer shares a screen, this is the order to walk through.

4.1 Ingest the ratings file with an explicit schema

```
ratings_schema = StructType([
    StructField("tconst", StringType(), True),
    StructField("averageRating", DoubleType(), True),
    StructField("numVotes", IntegerType(), True)])

ratings_df = (spark.read
    .option("header", True).option("sep", "\t")
    .schema(ratings_schema).csv(ratings_path))
```

- `sep = "\t"`: IMDb files are tab-separated, not comma-separated. `.csv()` is just Spark's delimited-text reader.
- **Explicit schema**: you tell Spark the column names and types instead of letting it scan the file to guess. That is faster and documents what you expect.
- Spark reads the **.gz** directly. Trap: gzip is not splittable, so each .gz file is read by a single task. Fine for 8.7 MB, slower for the 228 MB basics file.
- Nothing is read yet: **this line only defines the DataFrame** (lazy evaluation). The first *display()* triggers the read.

4.2 Bronze: persist what you ingested

```
ratings_df.write.format("delta").mode("overwrite").save(bronze_path)
```

Delta format gives you a transactional, re-readable copy. Later stages read *Bronze*, not the original gz, so you can inspect or rebuild Silver without touching the source. Be precise: Bronze here is the data *as read with the schema*, not a byte-for-byte copy of the file.

4.3 Silver: clean and validate

```
silver_ratings = (bronze_ratings
    .dropDuplicates(["tconst"])
    .filter(F.col("tconst").isNotNull())
    .filter(F.col("averageRating").between(0, 10))
    .filter(F.col("numVotes") >= 0)
    .withColumn("averageRating", F.col("averageRating").cast("double"))
    .withColumn("numVotes", F.col("numVotes").cast("long")))
```

Line	What it does	Detail worth knowing
`dropDuplicates`	Keeps one row per title ID	If duplicates disagreed on rating, Spark does not promise which one it keeps. A production rule would choose deliberately.
`isNotNull`	Removes rows with no title ID	Without a key a row cannot be joined later.
`between(0, 10)`	Keeps ratings from 0 to 10, inclusive	A NULL rating also fails this test, so null ratings are dropped too.
`numVotes >= 0`	Drops negative vote counts	Same: NULL votes are filtered out.
`cast`	Standardises types	Votes widen from 32-bit int to long. The cast cannot recover values already lost at read time.

VERIFIED
Profile of Silver from your output: **1,719,798 rows**, 1,719,798 unique IDs (no duplicates), mean rating **6.973**, maximum votes **3,250,919** (the Shawshank Redemption's count).

4.4 Gold: the weighted rating

```
C = silver.agg(F.avg("averageRating")).first()[0]            # 6.973
m = silver.approxQuantile("numVotes", [0.90], 0.01)[0]      # 485.0

gold = silver.withColumn("weightedRating",
    (v / (v + m)) * averageRating + (m / (v + m)) * C)
```

Both *first()* and *approxQuantile()* are actions: they run a Spark job immediately. Full explanation of the formula is in Section 5.

4.5 Movie metadata: filter, clean the missing marker, join

```
basics = (spark.read.option("header", True).option("sep", "\t")
    .schema(basics_schema).csv(basics_path)
    .filter(F.col("titleType") == "movie")
    .select("tconst", "primaryTitle", "startYear", "runtimeMinutes", "genres"))

basics_clean = (basics
    .withColumn("startYear", F.expr("try_cast(nullif(startYear, '\\\\N') AS INT)"))
    .withColumn("runtimeMinutes", F.expr("try_cast(nullif(runtimeMinutes, '\\\\N') AS INT)")))

movie_gold = gold.join(basics_clean, on="tconst", how="inner")
```

- **Why year and runtime are read as strings first**: IMDb writes a literal backslash-N for missing values, which would break an integer read.
- `nullif(x, marker)` returns NULL when x equals the marker. `try_cast` converts to int and returns NULL (instead of crashing) if the text is malformed.
- **Inner join**: only titles present in both ratings and movie metadata survive. That is how 1.7M rated titles become 349,988 rated movies.
- **Loose end to know about**: the *genres* column keeps the \N marker for movies without genres (one appears in your top-20 output). You handle it later by filtering, not at the join. Cleaning it to NULL here would be tidier.

4.6 Genre analysis with split and explode

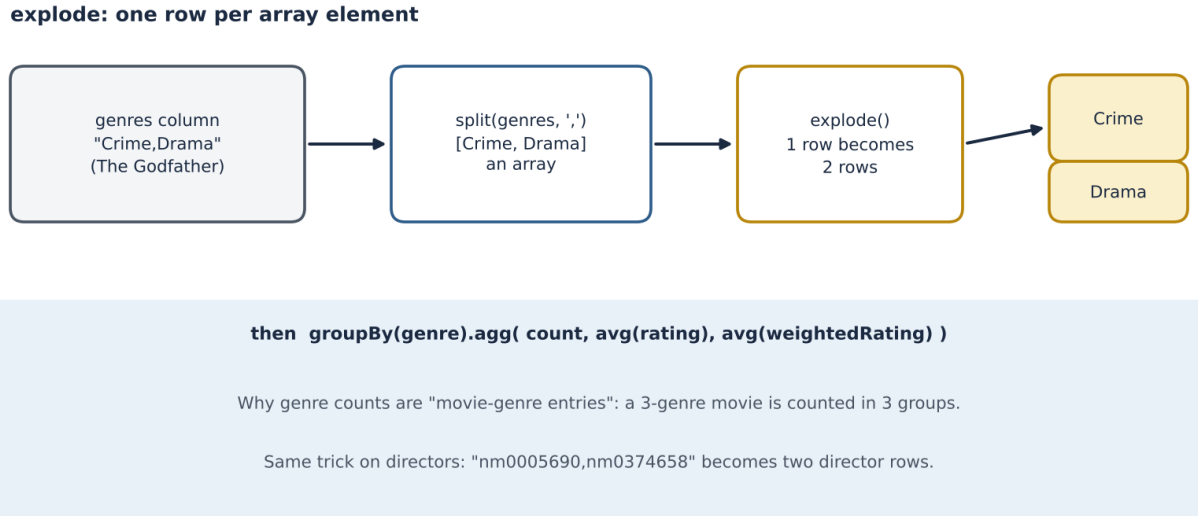


Figure 5. *How a comma-separated genre string becomes one row per genre before grouping.*

```
genre_stats = (eligible_movies
    .filter(F.col("genres").isNotNull())
    .withColumn("genre", F.explode(F.split("genres", ",")))
    .groupBy("genre")
    .agg(F.count("*").alias("movie_count"),
         F.round(F.avg("averageRating"), 2).alias("avg_rating"),
         F.round(F.avg("weightedRating"), 2).alias("avg_weighted_rating"))
    .filter(F.col("genre") != r"\N"))
```

4.7 Decades

`floor(startYear / 10) * 10` maps any year to its decade: floor(1997 / 10) = 199, then × 10 gives **1990**. Rows with a NULL year are removed first. The 2020s bucket is incomplete because the decade is still running.

4.8 Directors

```
director_stats = (eligible_movies
    .join(crew_df, on="tconst", how="inner")
    .filter(F.col("directors").isNotNull() & (F.col("directors") != r"\N"))
    .withColumn("director_id", F.explode(F.split("directors", ",")))
    .groupBy("director_id")
    .agg(F.countDistinct("tconst").alias("eligible_movie_count"),
         F.avg("averageRating"), F.avg("weightedRating"), F.sum("numVotes"))
    .filter(F.col("eligible_movie_count") >= 3))
```

- A film with two directors becomes two rows, so each director is credited with the film.
- `countDistinct("tconst")` guards against counting a title twice for the same director.
- **≥ 3 movies** is a project choice that stops one or two lucky titles from topping the list. It is not a statistical rule.
- Results use **director IDs (nm...)**, not names. Names live in IMDb's separate name.basics file, which this project does not load.

4.9 Register Gold as SQL tables

```
for table_name, path in gold_tables.items():
    df = spark.read.format("delta").load(path)
    df.write.format("delta").mode("overwrite").saveAsTable(f"workspace.default.{table_name}")
```

saveAsTable makes the data queryable by name in SQL cells and by other users. The earlier Delta paths in the volume are files; tables are catalog objects.

5. The weighted rating: your most distinctive idea

Interviewers love this part because it shows judgement, not just plumbing. Know the intuition, the formula, a worked example, and the limits.

The problem it solves

Sorting by raw average rating puts a film with a handful of perfect votes above a classic with millions of votes. A small sample is a noisy estimate. The weighted rating **pulls scores backed by few votes toward the overall average**, and leaves well-voted scores almost untouched.

The formula

$$\text{weightedRating} = (v / (v + m)) \times R + (m / (v + m)) \times C$$

Symbol	Meaning	Value in your run	Notebook name
R	This title's average rating	varies by title	`averageRating`
v	This title's number of votes	varies by title	`numVotes`
C	Average rating across all titles	6.973	`C`
m	Minimum-votes weight: the 90th percentile of vote counts	485	`m` / `voteThreshold`

Read it as: **the first weight, v/(v+m), is how much we trust the title's own rating; the second, m/(v+m), is how much we lean on the global average.** They always add to 1. With v = 0 you get C. As v grows far beyond m you approach R.

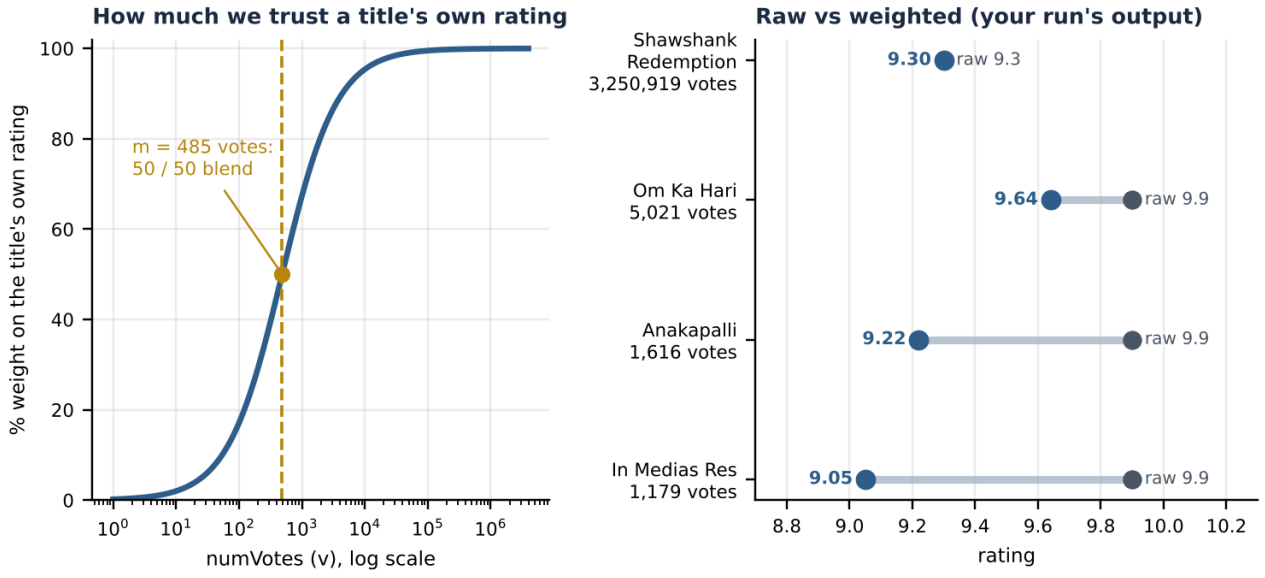


Figure 6. *Left: trust in a title's own rating rises with votes and is exactly 50% at 485 votes. Right: your actual output. A 9.9 backed by 1,179 votes falls to 9.05; the Shawshank Redemption barely moves.*

Worked example using your own numbers

Title	R	v	Weight on R = v/(v+485)	Calculation	Result
Om Ka Hari	9.9	5,021	0.912	0.912×9.9 + 0.088×6.973	**9.64**
Anakapalli	9.9	1,616	0.769	0.769×9.9 + 0.231×6.973	**9.22**
In Medias Res	9.9	1,179	0.709	0.709×9.9 + 0.291×6.973	**9.05**
The Shawshank Redemption	9.3	3,250,919	0.99985	≈ 9.3	**9.30**

All four results match the weightedRating values in your saved output.

Why the 90th percentile for m?

It lets the data pick the threshold instead of hard-coding a number. *approxQuantile* with relative error 0.01 estimates the vote count that 90% of titles fall below, which returned 485. It is a project choice, not an IMDb rule: a higher m shrinks low-vote titles harder.

Why also filter numVotes ≥ voteThreshold?

The weighted score already dampens low-vote titles. The filter adds an **eligibility rule** for ranking tables, so titles below 485 votes never appear in a top list, genre average, decade average or director count.

WATCH OUT

Caveat 1: m was computed over every rated title, not just movies. The 90th percentile of 1.72M titles (mostly episodes and shorts with few votes) is 485. For movies alone the bar is much easier to clear: about 72,000 of 349,988 movies (≈ 21%) pass it, not 10%. Honest answer: "The threshold is a reasonable starting point. If I rebuilt it, I would compute m on movies only, or raise it."

WATCH OUT: THE #1 QUESTION YOU WILL GET

Caveat 2: brand-new titles can top the list. In your output, 7 of the top 10 are 2026 releases with between 1,600 and 5,100 votes and raw ratings of 9.6 to 9.9. At m = 485 the score is only lightly shrunk. Early votes on new releases are often enthusiast-heavy. Fixes: a larger m (IMDb's Top 250 has historically required about 25,000 votes; check the current figure), a minimum release age, or a time-aware score.

SAY THIS

"Why is Om Ka Hari ranked above The Shawshank Redemption?"
"With 5,021 votes it clears the 485-vote threshold, so its 9.9 keeps about 91% of its weight and lands at 9.64. The weighted score only corrects for *tiny* samples, not for early enthusiastic voting on new releases. That is a limitation of a single global threshold, and I would address it with a higher movie-specific threshold or a release-age rule."

Is it IMDb's real formula?

Say: "It is a Bayesian-style weighted rating based on a well-known approach. I do not claim it reproduces IMDb's internal ranking." **Shrinkage toward the mean** is the statistical name for the effect: extreme scores backed by little evidence are pulled toward the average.

6. Results you can quote in the interview

Know two or three findings well. Each one below comes with the numbers from your saved output and the caution to attach.

Genres

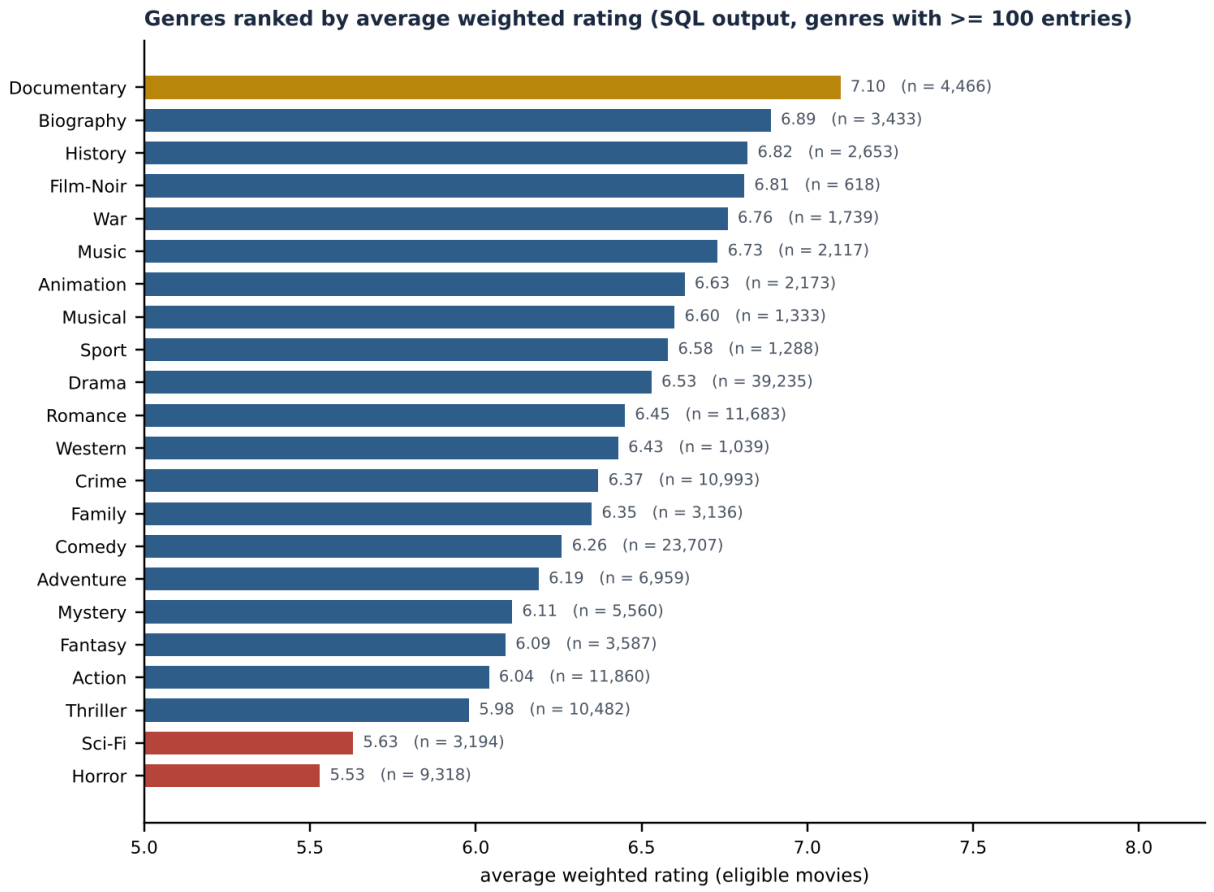


Figure 7. *Average weighted rating by genre. n is the number of movie-genre entries. Documentary is highest; Horror and Sci-Fi are lowest.*

- **Documentary** leads at 7.10 weighted. **Horror** (5.53) and **Sci-Fi** (5.63) are lowest.
- **Drama** (39,235) and **Comedy** (23,707) have the most entries. Entries sum to more than the movie count because a movie with three genres counts three times.
- Your PySpark genre table shows 26 genres; the SQL version shows 22. The SQL query's `HAVING COUNT(*) >= 100` drops News (60), Reality-TV (4), Talk-Show (2) and Adult (49).
- Caution: audience selection matters. People who vote on documentaries chose to watch them, so this describes the ratings given, not how good the genre is.

Decades

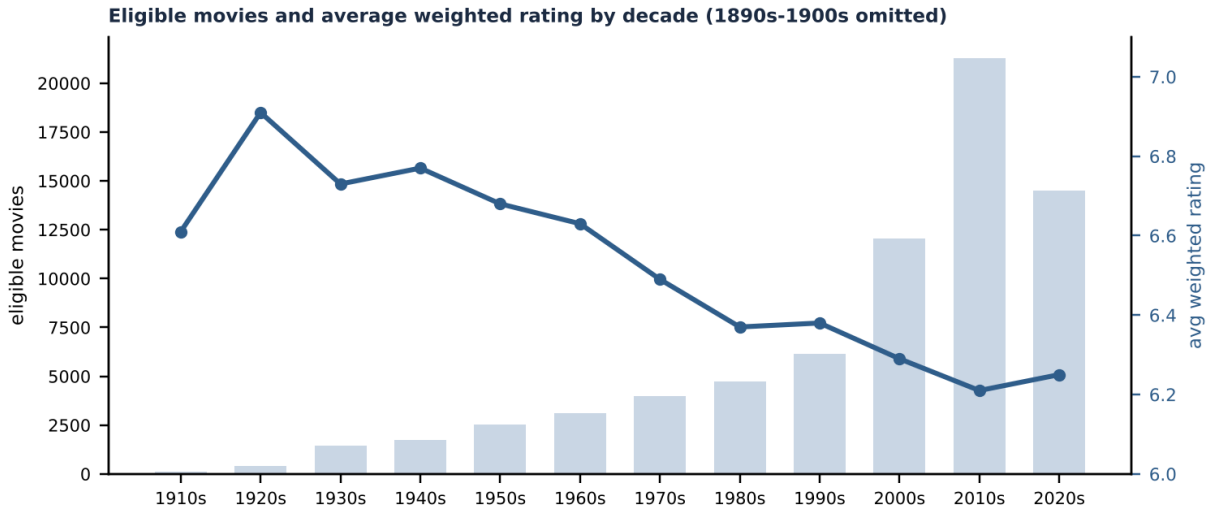


Figure 8. *Eligible movies per decade (bars) and average weighted rating (line). The 2020s are an incomplete decade.*

- Eligible movies per decade grow from 129 (1910s) to 21,298 (2010s). The average weighted rating drifts from about 6.9 (1920s) to about 6.2 (2010s).
- Likely reasons are survivorship (only well-regarded old films are still rated and watched) and more low-budget films being made and rated recently. Present those as hypotheses, not findings.
- The 1890s and 1900s have 1 and 2 movies, so those rows mean nothing statistically.

Top movies

#	Title	Year	Genres	IMDb	Votes	Weighted
1	Om Ka Hari	2026	Comedy, Drama	9.9	5,021	9.64
2	The Shawshank Redemption	1994	Drama	9.3	3,250,919	9.30
3	Anakapalli	2026	Action, Crime, Drama	9.9	1,616	9.22
4	The Godfather	1972	Crime, Drama	9.2	2,263,773	9.20
5	Anbil Avan	2026	Action, Drama, Romance	9.7	2,107	9.19
6	Oye Chill Maar	2026	Comedy, Drama	9.7	2,087	9.19
7	Manjar	2026	Thriller	9.6	2,455	9.17
8	Anireekshitha Atithigalu	2026	Drama, Thriller	9.8	1,646	9.16
9	Happy Journey	2026	Comedy, Drama	9.7	1,724	9.10
10	The Dark Knight	2008	Crime, Thriller	9.1	3,237,851	9.10

This is the table behind the Caveat 2 discussion in Section 5: classics and brand-new releases share the top ten.

Directors

Director ID	Eligible movies	Avg rating	Avg weighted	Total votes	Why it is interesting
nm0002031	85	6.68	6.80	1,093,341	Most eligible movies of anyone
nm0001238	70	4.71	5.44	96,057	Prolific but low-rated: quantity is not quality
nm0000406	67	6.82	6.92	770,924	High volume and above-average score
nm0001348	44	7.08	7.08	719,143	Best weighted score among the 20 highest-volume directors
nm0634240	13	8.18	8.18	19,252,360	Strong score with huge audience (19M votes)
nm0606835	3	8.73	8.48	18,833	Highest weighted score, but only 3 movies

- The SQL query ranks by **eligible movie count**, with average weighted rating as the tie-breaker. It does **not** identify the "best" director.
- A high average over a few eligible films can be luck. That is why the notebook requires at least 3.
- These are IDs. If asked for names, say you would join IMDb's name.basics file.

7. The SQL layer

Four queries on the registered Gold tables. Be able to explain one of them clause by clause, and to say what business question each answers.

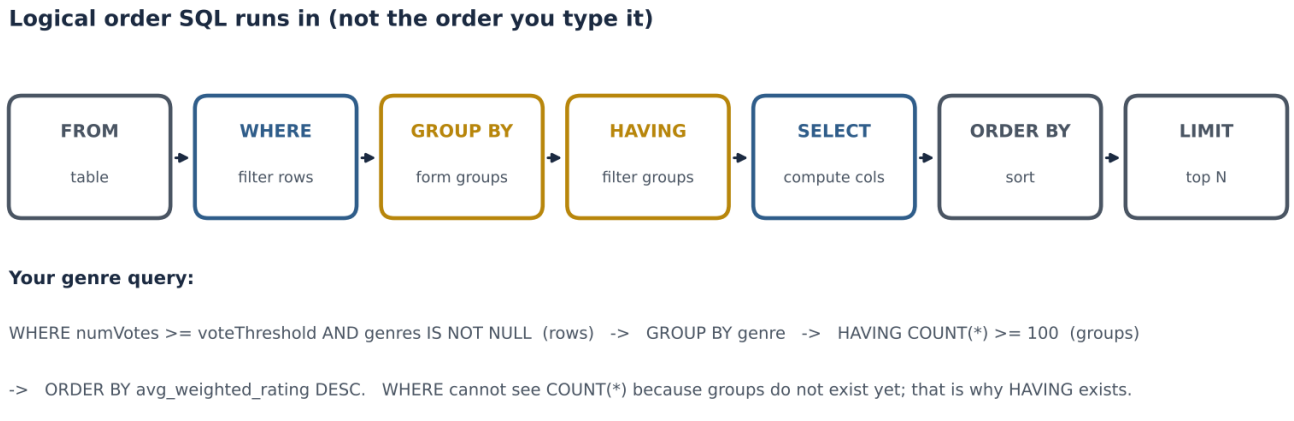


Figure 9. *SQL is written SELECT-first but logically runs FROM-first. This explains why WHERE and HAVING are different.*

Query 1: Top-ranked movies

```
SELECT primaryTitle, startYear, genres,
       ROUND(averageRating, 2) AS imdb_rating, numVotes,
       ROUND(weightedRating, 2) AS weighted_rating
FROM workspace.default.cinemascope_movie_rankings
WHERE numVotes >= voteThreshold
ORDER BY weightedRating DESC
LIMIT 20;
```

Question answered: which movies rank highest once vote count is accounted for? **WHERE** keeps only eligible movies (at least 485 votes). **ORDER BY ... DESC** sorts best-first. **LIMIT 20** keeps the first 20 after sorting. Note it orders by the unrounded weightedRating, while showing the rounded value.

Query 2: Genre comparison (the one to rehearse)

```
SELECT genre, COUNT(*) AS movie_count,
       ROUND(AVG(averageRating), 2) AS avg_rating,
       ROUND(AVG(weightedRating), 2) AS avg_weighted_rating
FROM (
    SELECT EXPLODE(SPLIT(genres, ',')) AS genre, averageRating, weightedRating
    FROM workspace.default.cinemascope_movie_rankings
    WHERE numVotes >= voteThreshold AND genres IS NOT NULL AND genres != '\\N'
) g
GROUP BY genre
HAVING COUNT(*) >= 100
ORDER BY avg_weighted_rating DESC;
```

Step	What it does
Inner SELECT	SPLIT turns "Crime,Drama" into an array; EXPLODE turns the array into one row per genre. WHERE removes ineligible movies, NULL genres and the \N marker.
Outer GROUP BY genre	Collects all rows with the same genre into one group.
COUNT, AVG	For each group: number of movie-genre entries, average raw rating, average weighted rating.
HAVING COUNT(*) ≥ 100	Filters *groups* after aggregation, dropping tiny genres. WHERE cannot do this because counts do not exist yet.
ORDER BY alias	Sorts by the computed column avg_weighted_rating, best first.

Query 3: Decades

```
SELECT FLOOR(startYear / 10) * 10 AS decade, COUNT(*) AS movie_count,
       ROUND(AVG(averageRating), 2) AS avg_rating,
       ROUND(AVG(weightedRating), 2) AS avg_weighted_rating
FROM workspace.default.cinemascope_movie_rankings
WHERE numVotes >= voteThreshold AND startYear IS NOT NULL
GROUP BY FLOOR(startYear / 10) * 10
ORDER BY decade;
```

Grouping by an *expression* is allowed: every year in 1990 to 1999 yields 1990, so each decade becomes one group. NULL years are excluded first.

Query 4: Directors by eligible movie count

```
SELECT director_id, eligible_movie_count, avg_rating, avg_weighted_rating, total_votes
FROM workspace.default.cinemascope_director_stats
ORDER BY eligible_movie_count DESC, avg_weighted_rating DESC
LIMIT 20;
```

No aggregation here: the aggregation was already done in PySpark and saved. **Secondary sort**: when two directors tie on movie count, the higher average weighted rating goes first.

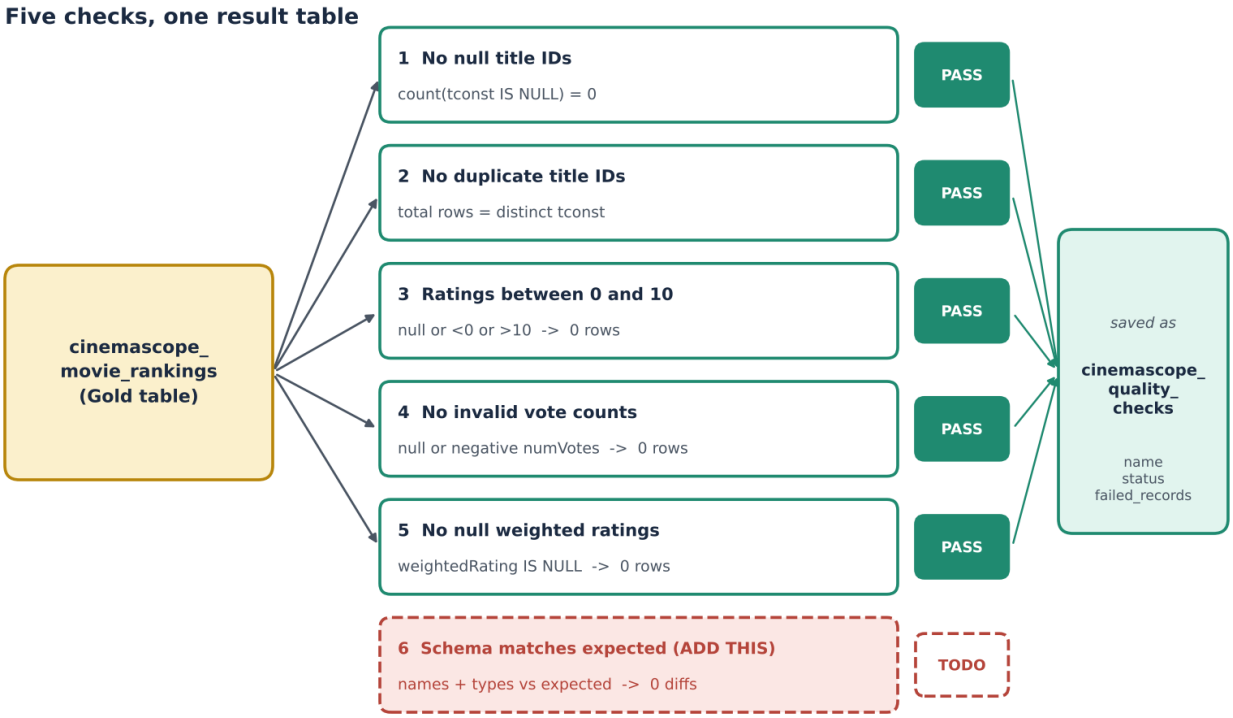
> **SAY THIS**
> **"Why use both PySpark and SQL?"** "PySpark builds and persists the curated datasets. SQL is the natural interface for analysts querying them. Both read the same underlying Delta data."

Spark SQL functions you used

Function	Meaning	Example from your queries
`SPLIT(s, ',')`	String to array by delimiter	genres "Crime,Drama" → [Crime, Drama]
`EXPLODE(array)`	Array to one row per element	One row per genre
`FLOOR(x)`	Round down to integer	FLOOR(1997/10) = 199
`ROUND(x, 2)`	Round to 2 decimals	weighted_rating 9.6421 → 9.64
`COUNT(*) vs COUNT(col)`	Rows vs non-NULL values	COUNT(*) counts every row in a group

8. Data quality: exactly what you check, and what you do not

This is where interviews get specific. The five checks are real and persisted. Know their exact logic, what they are good for, and the gap around "schema consistency".



The dashed row is the check missing for the "schema consistency" claim. See Section 8 for ready-to-paste code.

Figure 10. *The five implemented checks, plus the sixth (dashed) that would make the schema-consistency claim true.*

#	Check	Logic in your code	Catches	Result
1	No null title IDs	sum(when(tconst is null, 1)) = 0	Rows that could not be joined	PASS, 0
2	No duplicate title IDs	total_rows = countDistinct(tconst); failed = difference	Repeated IDs, for example from a bad join	PASS, 0
3	Ratings between 0 and 10	null or < 0 or > 10 counted as invalid	Missing or out-of-range ratings	PASS, 0
4	No invalid vote counts	null or negative numVotes counted	Missing or negative votes	PASS, 0
5	No null weighted ratings	weightedRating is null counted	Score not computed for a row	PASS, 0

How to talk about them honestly

- **They run on the final Gold movie table**, after Silver has already filtered bad rows. So today they can only fail if something upstream breaks (a bad join, a removed filter). Describe them as **regression guards on the published table**. That is a legitimate and valuable role.
- **Duplicate check nuance:** countDistinct ignores NULLs, so a null ID would not appear as a duplicate. That is why the null check exists separately.
- **PASS only means the rule held.** It does not certify that ratings are accurate or the sample is representative.
- **Results are saved** to `cinemascope_quality_checks` so they can be queried. A production version would add run ID, timestamp and table version.

- **Nothing stops the pipeline on FAIL.** The notebook records status but does not raise an error. A fail-fast version would assert that no check failed before publishing.

What is not checked yet

Gap	Why it matters	Simple fix
Schema (names and types)	Source changes would go unnoticed. This is the resume gap.	Check 6 below
Rows lost in the inner join	1.7M rated titles become 349,988; you never measured unmatched keys	left_anti join count, or compare row counts before and after
Missing-marker leftovers	genres can still contain \N	Convert \N to NULL for genres in basics_clean
Plausible year and runtime	Out-of-range years or zero runtimes pass silently	Range checks on startYear and runtimeMinutes
Volume and freshness	A truncated file would still PASS	Row-count bounds and source file timestamp

Add the schema check (ready to paste)

Put this in a new cell after the "Calculate Data Quality Metrics" cell and before "Save Data Quality Check Results". **I could not run it against your Databricks workspace, so run it and check the output before you claim it.** The expected types below come from your notebook's own output schemas.

```
# Expected schema of the Gold movie table (from your pipeline)
expected_schema = {
    "tconst": "string", "averageRating": "double", "numVotes": "bigint",
    "weightedRating": "double", "voteThreshold": "double",
    "primaryTitle": "string", "startYear": "int",
    "runtimeMinutes": "int", "genres": "string",
}

actual_schema = {f.name: f.dataType.simpleString() for f in movies.schema.fields}

missing    = [c for c in expected_schema if c not in actual_schema]
wrong_type = [c for c, t in expected_schema.items()
              if c in actual_schema and actual_schema[c] != t]
extra      = [c for c in actual_schema if c not in expected_schema]  # informational

schema_failures = len(missing) + len(wrong_type)
print("missing:", missing, "| wrong type:", wrong_type, "| extra:", extra)

schema_row = spark.createDataFrame(
    [("Schema matches expected", "PASS" if schema_failures == 0 else "FAIL", schema_failures)],
    ["check_name", "status", "failed_records"])
quality_df = quality_df.unionByName(schema_row)   # then re-run the save cell
```

TIP

After it runs, the report shows six checks. You can then truthfully say "I check null IDs, duplicate IDs, rating range, vote counts, null scores and schema conformance." Note the resume says *five* checks; if you add the sixth, either update the number to six or keep it as a bonus you can mention verbally.

Quick answers

- **Data-quality failure vs pipeline failure:** a pipeline failure means the job could not finish (file missing, write failed). A quality failure means the job finished but the data broke a rule.
- **Explicit schema vs schema check:** declaring a schema tells Spark how to read. A schema check *verifies* the result matches. They are different things.

9. Terminology cards

If the interviewer asks "what is X?", give the plain-English sentence first, then the "In CinemaScope" sentence. Both together show you understand it and have used it.

Platform and storage

Term	Plain English	In CinemaScope
Apache Spark	A distributed engine that splits data across machines and processes the pieces in parallel.	The engine that runs every transformation.
PySpark	Python interface to Spark.	All DataFrame code is PySpark.
Databricks	A managed platform that runs Spark and adds notebooks, governance and storage features. Spark is the engine; Databricks is the platform.	Where the notebook runs and where tables live.
Delta Lake	A storage layer that adds transactions, versioning and schema enforcement on top of data files (Parquet files plus a transaction log).	Every layer is written with format('delta').
Delta table	A table stored in Delta format and queryable by name.	workspace.default.cinemascope_* tables.
Unity Catalog	Databricks' governance layer: catalogs, schemas, tables, volumes and permissions.	Names like workspace.default.cinemascope_ratings.
Volume	A governed location for *files* (not tables) in Unity Catalog.	cinemascope_raw holds the IMDb files and the layer folders.
Data lake / warehouse / lakehouse	Lake: cheap flexible file storage. Warehouse: curated, structured, fast SQL. Lakehouse: lake storage with warehouse-style reliability.	Delta on Databricks is the lakehouse pattern.
TSV and .gz	Tab-separated text; gzip is a compression format.	IMDb ships .tsv.gz files.

Pipeline concepts

Term	Plain English	In CinemaScope
ETL / ELT	Extract, transform, load. ELT loads first and transforms inside the platform.	Read → transform → write: ETL-style in a notebook.
Batch vs streaming	Batch: process a bounded chunk per run. Streaming: process events continuously.	Batch: bulk IMDb files.
Medallion architecture	Bronze (ingested), Silver (cleaned), Gold (curated) layers.	Ratings pass through all three.
Schema / explicit schema	A schema lists column names and types. Explicit means you declare it instead of letting Spark infer.	ratings_schema, basics_schema, crew_schema.
Schema enforcement vs schema drift	Enforcement rejects writes that do not match the table. Drift is unexpected change in the source's columns or types.	Not tested here; Section 8 adds a check.
Idempotency	Re-running with the same input gives the same result with no extra side effects.	Overwrite makes reruns repeatable for fixed input.
Overwrite vs append	Overwrite replaces the target; append adds to it.	Every write uses overwrite (full refresh).
Data profiling	Summarising counts, ranges, distinct values to understand data.	The Silver profile cell.
Join key / cardinality	The column used to match rows; cardinality is how many rows match per key (1:1, 1:many).	tconst; one row per title in each file.

Term	Plain English	In CinemaScope
Lineage	The trail showing where each dataset came from.	Source → Bronze → Silver → Gold, as in the Section 3 pipeline diagram.

Spark internals

Term	Plain English	In CinemaScope
DataFrame	Distributed table with named, typed columns.	ratings_df, basics, crew_df, gold.
Transformation	An operation that defines a new DataFrame (filter, select, join, withColumn, groupBy).	Nearly every line in the pipeline.
Action	An operation that makes Spark actually run (display, count, first, write).	display(), save(), approxQuantile().
Lazy evaluation	Spark waits until an action, then optimises and runs the whole plan.	Diagram below: the spark.read line alone does nothing.
Shuffle	Moving data between machines so matching keys end up together; needed for joins and groupBy and expensive.	Joins on tconst and the groupBy aggregations.
Partition	A chunk of data processed by one task.	Not tuned in this notebook.
Broadcast join	Copy a small table to every machine to avoid a shuffle.	Not used; a possible optimisation.
approxQuantile	Fast estimate of a percentile within an error tolerance.	90th percentile of numVotes = 485.
explode / split	split makes an array from a string; explode makes one row per array element.	Genres and directors.
Aggregation	Reducing many rows to a summary: count, avg, sum, max.	Genre, decade and director stats.

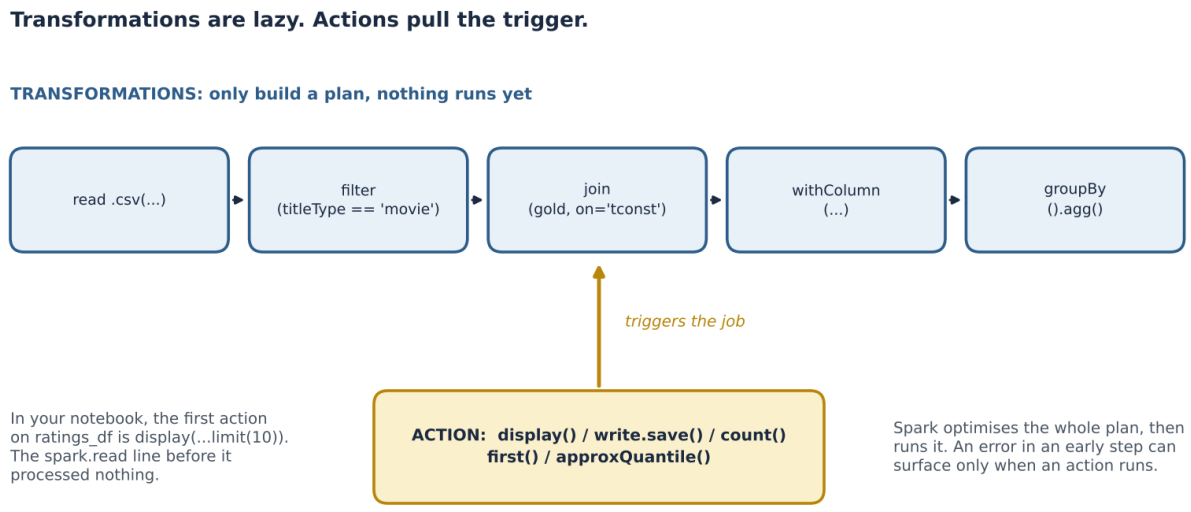


Figure 11. *Transformations build a plan; an action triggers the job.*

Data quality

Term	Plain English	In CinemaScope
Data quality dimensions	Completeness (nothing missing), uniqueness (no repeats), validity (allowed values), consistency (same format), accuracy (true to reality), timeliness (fresh enough).	Your checks cover completeness, uniqueness, validity.
NULL / missing marker	NULL means no value. IMDb writes a literal \N instead.	nullif converts \N into NULL.
Referential integrity	Every key in one table has a matching row in the related table.	Not checked; inner join silently drops non-matches.
Quarantine	Putting bad rows aside with a reason instead of deleting them.	Not implemented; Silver just filters.

SQL and statistics

Term	Plain English	In CinemaScope
WHERE vs HAVING	WHERE filters rows before grouping; HAVING filters groups after aggregation.	HAVING COUNT(*) >= 100.
Percentile / quantile	The value below which a given share of observations fall.	p90 of votes = 485.
Weighted average	An average where some values count more than others.	weightedRating blends R and C.
Bayesian shrinkage	Pulling noisy estimates toward a prior (here, the global mean) when evidence is thin.	Why a 9.9 on 1,179 votes becomes 9.05.
Survivorship bias	Judging a group by only the members that survived.	Old decades look better because only enduring films remain.

10. Question bank by theme

Models, not scripts: say them in your own words. Each answer starts direct, then gives the reason, then points at your code. "Follow-up" lines are the next question an interviewer is likely to ask.

A. The project and its design

Q1 **What problem does CinemaScope solve?**

It turns raw IMDb bulk files into structured, validated tables that answer questions like: which movies rank highest after accounting for vote count, how ratings differ by genre and decade, and how directors compare.

Follow-up: *What is the output?* Gold Delta datasets and SQL tables for rankings, ratings and director stats, plus a quality-results table.

Q2 **Why IMDb data?**

IMDb publishes ratings, title metadata and crew as bulk files with a natural shared key. They contain real data-engineering problems: compressed TSV, a missing-value marker, type conversion, joins and aggregation.

Follow-up: *Limitations?* It covers only the selected files, and ratings are user votes, not an objective measure of quality.

Q3 **What does "end-to-end" mean here?**

Ingestion, transformation, layered storage, quality checks, curated outputs and SQL analysis in one workflow. I do not use it to imply scheduling or monitoring, which the notebook does not include.

Follow-up: *Is it production-ready?* No. It would need orchestration, alerting, retries, access control, tests and incremental loads.

Q4 **Why batch and not streaming?**

The source is a set of bulk files published periodically, so a bounded run per refresh is the natural fit. Streaming would add continuous processing, state and checkpointing that this data does not need.

Q5 **Why PySpark rather than pandas?**

PySpark scales across a cluster and its DataFrame API handles large joins and aggregations. pandas is simpler and often faster for data that fits one machine. The ratings file has 1.7M rows, where either could work; Spark was my chosen engine, and the 228 MB basics file benefits from it.

Follow-up: *Did you benchmark?* No, so I do not claim a speedup.

Q6 **What is Databricks' role? Is it the same as Spark?**

No. Spark is the distributed processing engine. Databricks is the platform that runs it and adds notebooks, Unity Catalog, volumes and Delta tooling. My notebook uses Spark APIs for processing and Databricks for storage, tables and execution.

Q7 **Why save intermediate layers instead of chaining in memory?**

Stage boundaries make debugging easier, because I can inspect Bronze, Silver and Gold separately, and later stages reuse stable data. The cost is extra storage and I/O.

Q8 **Why do basics and crew not have Bronze and Silver copies?**

Ratings were the core dataset, so I layered them fully. Basics and crew are read with explicit schemas and cleaned at the point of joining. If I extended the project I would persist all three sources through every layer for consistency.

B. PySpark mechanics

Q9 **What happens in the Silver layer?**

I read Bronze, drop duplicate title IDs, remove null IDs, keep ratings between 0 and 10, drop negative votes, standardise types, and save as Delta. Profiling then shows 1,719,798 rows, all unique.

Follow-up: *Does dropDuplicates pick a deterministic row?* Not when duplicates differ on other columns. A production rule should define which to keep, for example by timestamp.

Q10 **Why use explicit schemas?**

They define expected columns and types at read time, avoid an inference pass and document the contract. They do not prove the data conforms; malformed values still need handling and validation.

Follow-up: *What could go wrong?* Columns may change, values may be malformed, or the \N marker may sit in a numeric column.

Q11 **Why cast numVotes to long when you read it as integer?**

Long gives a wider range and a consistent downstream type. Honestly, the cast happens after the read, so it cannot rescue a value that overflowed at ingestion. Reading it as LongType at ingestion would be cleaner. Max votes is 3,250,919, well inside int range.

Q12 **Transformation versus action?**

A transformation defines a new DataFrame (filter, select, join, withColumn, groupBy). An action triggers execution (display, count, write, first). In my notebook display and write cells, first() and approxQuantile() trigger work.

Follow-up: *Lazy evaluation, in one line?* Spark builds a plan and only runs it, optimised, when an action needs a result.

Q13 **Why can an error appear later than the line that caused it?**

Because of lazy evaluation: a faulty transformation is not executed until an action runs the plan.

Q14 **Explain filter, withColumn, groupBy and agg with examples.**

filter keeps rows that match a condition (rating between 0 and 10). withColumn adds or replaces a column (weightedRating). groupBy sets the grouping keys (genre). agg computes aggregates per group (count, avg, sum).

Q15 **What do split and explode do, and can they change row counts?**

split turns "Crime,Drama" into an array; explode makes one row per element. Yes, row counts rise on purpose: a movie with three genres yields three rows.

Follow-up: *What about NULL or the \N marker?* Filter or normalise first, or \N would be counted as a genre.

Q16 **Why an inner join, and when would you use a left join?**

A ranked movie needs both a rating and metadata, so inner is correct, but it silently drops non-matches. A left join preserves every row from the left side when completeness matters more.

Follow-up: *Can a join multiply rows?* Yes if keys repeat on either side. tconst is unique in each IMDb file, so counts did not inflate; the duplicate check at Gold would reveal a problem.

Q17 **count versus countDistinct?**

count(*) counts rows; countDistinct(tconst) counts distinct non-null IDs. Comparing them exposes duplicate IDs, while NULLs need their own check because countDistinct ignores them.

Q18 **What does approxQuantile do?**

It estimates a percentile of a column within an error tolerance, faster than an exact computation on big data. I asked for the 90th percentile of numVotes with relative error 0.01, and it returned 485.0.

Q19 **What is a shuffle, and how would you reduce its cost?**

A shuffle redistributes data across partitions, needed for joins and groupBy, and costs network and disk I/O. I would inspect the plan and Spark UI, filter and select early, tune partitioning, and broadcast a small table in a join. None of this is tuned in the notebook.

C. Delta Lake and Databricks

Q20 **What is Delta Lake and why use it?**

A storage layer over data files that adds ACID transactions, a transaction log and schema enforcement. I use it for reliable writes and re-reads of each layer.

Follow-up: *Did you use time travel or MERGE?* No, so I do not claim them.

Q21 **Explain Bronze, Silver and Gold simply.**

Bronze is data as ingested, Silver is cleaned and validated, Gold is shaped for use. They are conventions, and each project adapts them.

Q22 **Is Bronze untouched raw data?**

It is raw relative to business rules, but it is written after reading with a declared schema, so it is not a byte-for-byte copy of the gz file.

Q23 **What is a Unity Catalog volume, and what does saveAsTable do?**

A volume is a governed location for files; I keep the raw IMDb files and layer folders there. saveAsTable registers a dataset as a named catalog table so SQL can query it directly.

Q24 **What does mode('overwrite') mean, and what is the risk?**

It replaces the target contents on each run, which keeps full refreshes simple and repeatable. The risk in production is a partial failure or losing data that should have been kept; incremental writes need deliberate design.

Q25 **Delta schema enforcement versus your data-quality checks?**

Enforcement rejects writes that do not match the table's schema. My checks test business rules on values such as rating range and uniqueness. They are complementary.

D. Ratings logic

Q26 **Why a weighted rating instead of sorting by averageRating?**

A title with few votes can have an extreme average that is unreliable. The weighted rating blends the title's rating with the global average, trusting the title more as its votes grow.

Q27 **Explain the formula in plain English.**

v/(v+m) is the weight on the title's own rating; m/(v+m) is the weight on the global average C. They sum to 1. At v = 0 the score is C; at v far above m it approaches R. At v = 485 it is a 50/50 blend.

Q28 **Why the 90th percentile for m, and what are its limits?**

It sets the threshold from the data. It returned 485. It was computed over all rated titles, not movies only, so it is easy for movies to clear; and one global m ignores release age and genre. It is a project choice, not a universal rule.

Q29 **Is this IMDb's exact formula?**

No claim. It is a Bayesian-style weighted rating based on a well-known approach, implemented with my own threshold.

Q30 **Why also filter numVotes ≥ voteThreshold in ranking queries?**

The weighted score dampens low-vote titles; the filter adds an eligibility rule so tiny-vote titles cannot appear in top lists or skew genre, decade and director averages.

Q31 **Why require at least 3 eligible movies per director?**

To avoid ranking a director on one or two titles. Three is a project threshold and I would justify it as such, not as a statistical rule.

E. SQL and interpretation

Q32 **Explain GROUP BY and aggregates using your genre query.**

GROUP BY genre forms one group per genre; COUNT and AVG then summarise each group: number of entries, average rating, average weighted rating.

Q33 **Why HAVING instead of WHERE for the genre count?**

WHERE filters rows before grouping; counts do not exist yet. HAVING COUNT(*) >= 100 filters groups after aggregation.

Q34 **Why FLOOR(startYear / 10) * 10 for decades?**

Dividing by ten, flooring and multiplying by ten maps 1997 to 1990. NULL years are excluded first.

Q35 **Why do genre counts exceed the number of movies?**

A movie with several genres is counted once per genre after split and explode. The counts are movie-genre entries, not unique movies.

Q36 **Does the director query find the best director?**

No. It ranks by number of eligible movies with weighted rating as a tie-breaker. It is descriptive and does not control for genre, era, budget or audience.

Q37 **Why use SQL after PySpark?**

PySpark builds and persists the curated data; SQL is the convenient, widely-known interface for querying it. Both use the same registered Delta tables.

F. Data quality

Q38 **Name your five checks.**

Null title IDs; duplicate title IDs; ratings null or outside 0 to 10; vote counts null or negative; null weighted ratings. All are computed in one aggregation, saved as PASS/FAIL with a failed-record count.

Q39 **Does the notebook check schema consistency?**

Not as a test. I declare explicit schemas at ingestion and standardise types in Silver, but I do not compare the actual schema to an expected one. I would add that check, which is a few lines using the DataFrame's schema.

Q40 **Are five checks enough for production?**

They are good basics. I would add expected-column and type checks, accepted-value checks, referential-integrity counts after joins, volume and freshness checks, and alerting that fails the run.

Q41 **What does PASS mean?**

Only that the defined rule held on the current table. It does not certify the data is correct or representative.

Q42 **How could the duplicate check be better?**

Assert non-null keys first, then compare row count with distinct non-null keys and report each number separately. Currently countDistinct ignores NULLs, which the separate null check compensates for.

Q43 **Why persist quality results?**

They become queryable and reviewable instead of vanishing with notebook output. A production version would add run ID, timestamp and table version.

G. Scale, reliability and production

Q44 **What if the data grew 100 times?**

I would measure before changing code: partition counts, shuffle-heavy steps and the Spark UI. Then select only needed columns, filter early, consider partitioning and broadcast joins, and reconsider full overwrites in favour of incremental loads.

Q45 **A join unexpectedly doubles your row count. What do you do?**

Count duplicate keys on both sides and compare counts before and after. One-to-many may be valid; otherwise it signals duplicates. I would not blindly drop duplicates without a business rule.

Q46 **How would you make it idempotent?**

Overwrite makes reruns repeatable for fixed input. For incremental data I would use stable input versioning, deterministic transforms and key-based upserts (MERGE) so a rerun cannot create duplicates.

Q47 **How do you handle a malformed input file?**

Validate presence and expected columns, fail clearly if unusable, and route bad rows to a quarantine table with a reason when recoverable. My notebook uses explicit schemas but has no quarantine.

Q48 **How would you test this pipeline?**

Unit tests for the weighted-rating formula with hand-calculated values; cleaning-rule tests with nulls, duplicates and out-of-range values; join cardinality tests; explode tests; SQL aggregates on small fixtures; and an integration test that expected tables and checks exist after a run.

Q49 **How would you monitor and alert?**

Track run time, input file versions, row counts per stage, rejected rows and quality results. Alert on missing inputs, large row-count changes and failed checks.

Q50 **How would you handle schema drift?**

Define the expected schema, compare it on every run, and decide to fail, quarantine or accept compatible additions. Log the schema version so consumers stay compatible.

Q51 **What would you add to make it production-grade?**

Parameterised paths and run dates, orchestration, logging and metrics, retries and failure notifications, schema versioning, idempotent or incremental writes, run-level quality metadata, permissions and secrets management, tests and CI/CD.

11. Trap questions and cross-questions

These come from reading your actual outputs. Each is a place where an alert interviewer could probe. Have the answer ready and the confidence to say what you would improve.

If they ask...	A strong answer
"Why is a 9.9-rated 2026 film number one?"	Its 5,021 votes clear the 485-vote bar, so the score only shrinks to 9.64. The formula corrects tiny samples, not enthusiastic early voting. I would raise the threshold, compute it on movies only, or add a release-age rule.
"Which data did you compute m from?"	All 1.72M rated titles, including episodes and shorts. That is why about 21% of movies clear it instead of 10%. A movie-only threshold would be stricter.
"Can your quality checks ever fail?"	On the current pipeline, only if upstream logic breaks (a bad join, a removed filter), because Silver already filters bad rows. They guard the published table against regressions. Today they record status but do not halt the run; I would add a fail-fast step.
"Where is your schema consistency check?"	I enforce explicit schemas at read and cast types in Silver, but I have not built a test comparing actual to expected. I wrote one (Section 8) that compares column names and types and records PASS or FAIL.
"You say Bronze, Silver, Gold, but basics and crew skip them."	Correct. Ratings go through all three layers; metadata and crew are read with explicit schemas and joined at Gold. A fuller design would layer all sources.
"How many movies did the inner join drop?"	I did not measure that. I would count unmatched keys with a left_anti join. The big drop from 1.7M to 349,988 is expected because ratings include non-movie title types.
"Which duplicate does dropDuplicates keep?"	Spark does not guarantee which one. IMDb's ratings file has one row per title, so it does not matter here (0 duplicates found), but production needs an explicit rule.
"Drama has 39,235 entries; how many movies is that?"	Entries are movie-genre pairs, not movies. The total of about 72,000 eligible movies is smaller than the sum across genres because multi-genre films repeat.
"Some genres look like \N. Is that handled?"	Yes downstream: the Python query filters genre != \N and the SQL query excludes genres = \N. Cleaning it to NULL at the join would be tidier, and I would do that.
"Why try_cast and nullif?"	nullif turns IMDb's \N marker into NULL; try_cast converts to int and returns NULL for malformed text instead of failing the job.
"Why store Delta in a volume path and also as tables?"	The volume holds the layer files; I register the Gold outputs as tables for SQL access. In production I would register every layer as a governed table.
"Is overwrite safe?"	For a repeatable full refresh of a project, yes. In production a failed run could leave a partial result, so I would use atomic table operations and incremental logic.
"Why do you only show directors as IDs?"	I did not load IMDb's name.basics file. Joining on nm-ID gives names; it was out of scope for the ranking logic.
"Does a high weighted rating prove a better director?"	No. It is descriptive and ignores genre, era, budget and audience composition. Three or more eligible movies reduce luck but do not remove it.
"What is the difference between a pipeline failure and a data-quality failure?"	Pipeline failure: the job cannot complete (file missing, write error). Quality failure: the job completes but data breaks a rule. A robust system records both and decides which stops the run.
"What would you improve first?"	Add the schema check, measure join loss, move m to a movie-only threshold, add run metadata and fail-fast behaviour, then orchestration and incremental loads.
"I don't know" moments	Say what you did and did not test: "I haven't benchmarked that. My hypothesis is X, and I'd confirm it from the execution plan and stage timings." That beats inventing a result.

12. Revision card and pre-interview checklist

One page to read in the last ten minutes.

Numbers to know cold

Fact	Value	Fact	Value
Rated titles in Silver	1,719,798	Mean rating C	6.973
Movie records (final)	349,988	Vote threshold m (p90)	485
Eligible movies (approx.)	~72,000	Max votes	3,250,919
Source files	ratings 8.7 MB, basics 228 MB, crew 84 MB	Director minimum	≥ 3 eligible movies
Genre HAVING threshold	≥ 100 entries	Quality checks	5, all PASS
Most eligible movies (one director)	85	Top genre / bottom genre	Documentary 7.10 / Horror 5.53

The five checks, in order: (1) no null title IDs, (2) no duplicate title IDs, (3) ratings between 0 and 10, (4) no invalid vote counts, (5) no null weighted ratings.

Flow in one line

SAY THIS

IMDb TSV.GZ → **Bronze** (ratings as Delta) → **Silver** (dedupe, validate, cast) → **Gold** (weighted rating, join metadata and crew, genre/decade/director stats) → **SQL tables** → **quality checks**. Key: tconst. Formula: (v/(v+m))·R + (m/(v+m))·C.

Structure of a good answer: direct answer in one sentence, the reason, the line of code that shows it, then one honest limitation.

Checklist before the interview

	Task	Done when
☐	Decide the schema-consistency fix	You ran Section 8's cell and the report shows six checks, or the resume says "explicit ingestion schemas".
☐	Decide the check count wording	Resume says five, six or "five core checks" consistently with the notebook you show.
☐	Rehearse the pitch and the formula	30 and 90-second pitch without notes; compute 9.64 for Om Ka Hari from R, v, m, C.
☐	Know both caveats on m, draw the pipeline	"Computed on all titles" and "new releases can top the list"; sketch the flow in under a minute.
☐	Re-read the "do not claim" list in Section 2	No scheduler, streaming, alerting, MERGE or benchmarks claimed.

TIP

Everything in this guide is traceable to a cell in your notebook. If an interviewer asks "where is that?", open the matching section and point to it.